On Time, Events, Objects, and Relations in Object-Centric Event Data

No Author Given

No Institute Given

Abstract. Data in process mining (PM) is inherently temporal. Yet, existing object-centric event data (OCED) models, such as OCEL and Event Knowledge Graphs, attach time only to events, while objects and relationships remain static. Existing extensions target isolated aspects (e.g., evolving attributes or states), but the field lacks a systematic approach to represent time-varying objects and properties, including notably relationships. Since existing data models do not allow expressing dependencies, such as disallowing a *Borrow Book* event when a library is closed or a book is already on loan, analysts must rely on implicit or ad-hoc assumptions, which may lead to inconsistent analysis. To address this limitation, this paper introduces *Temporal Object Centric Event Data* (T-OCED), a temporal data model built on temporal property graphs that explicitly represents time for events as well as objects, attributes, and relationships. First, we formalize T-OCED and define well-formedness criteria aligning event occurrences with the temporal validity of referenced elements. Second, we show how T-OCED enables formalizing and verifying standard constraints (e.g., cardinalities) and semantic constraints, ensuring semantic consistency between events and the observed changes in objects and relationships. Third, we assess feasibility by realizing the model in a graph database system (Neo4j) and testing it on a simulated library dataset containing deliberately injected violations.

Keywords: Process Mining · Temporal Data · Data Modeling · Consistency Constraints.

1 Introduction

Object-centric process mining aims at providing models and methods for studying process behavior in terms of the co-evolution of multiple interacting objects (in possibly complex relations) instead of event sequences alone [29,6]. To support this aim, several representations for event data over objects have been proposed, including OCEL 1.0 event tables [16] and Event Knowledge Graphs (EKGs) [11].

However, existing data representations still offer limited support for describing the temporal evolutions of objects and the relations themselves in relation to events. For example, in a library setting, members may borrow multiple books at the same time, provided that these books have not been reserved by others at the moment of borrowing. While events carry timestamps, objects, attributes

and relationships are typically modeled as static, and their changes over time are not explicitly represented. Even though their evolution may be sometimes derivable through analysis techniques, it cannot be reconstructed in general [12], unless explicit, external domain knowledge is provided [7]. A natural alternative would then be to directly represent information on the temporal evolution of objects and relations explicitly. However, neither EKGs or OCEL event tables can do this systematically while OCEL2.0 [20] only records attribute evolution.

This lack of temporal modeling leads to several issues, such as wrong statistics and misleading control-flow dependencies: the very problems object-centric process mining aims to address. Consider a library, where a member may cease being a member *only if they returned all their borrowed books and still have a valid membership*. Since the relationships between a member and their borrowed books, as well as their membership status, evolve over time, treating them as static would impede correctly discovering how books and memberships synchronize, and skews statistics such as the average number of simultaneously borrowed books per member – key factors in object-centric process discovery [5,27,30].

As a result, analyses rely on implicit assumptions or ad-hoc bottom up extensions to deal with time-varying concepts. Many approaches implicitly assume that relationships are rigid and thus valid throughout the entire observation window (see the discussion in [27]), or that an object is created by the first event mentioning it, and destroyed by its last [24]. More recent work extends OCEL1.0 with time-varying attributes (cf. OCEL2.0 [20]), and evolving object states [21].

In this paper, we propose to tackle this problem from a different angle. Inspired by the long-standing literature in temporal data modeling [4], we extend *temporal property graphs* (TPGs) [3], which model evolving objects, attributes, and object-to-object (O2O) relationships, with events and event-to-object (E2O) relationships resulting in *Temporal Object Centric Event Data* (T-OCED). Our contribution is threefold. First, we formalize T-OCED as a specialization of TPGs according to OCED [12] aligning event occurrences with the temporal validity of objects, attributes, and relationships. Second, we show how constraints can be formulated over T-OCEDs to ensure interpretability and consistency, from standard snapshot constraints (enforced at each time point) to richer semantic one. In particular, we express semantic constraints on the proper semantics of event types to ensure that every occurrence of an event type manipulates the underlying data consistently (e.g., any *Borrow book* event creates a *Borrowed By* relationship). Third, we demonstrate feasibility through an encoding of T-OCED in Neo4j, and show that semantic inconsistencies can be effectively detected using Cypher queries on a simulated dataset with controlled violations.

The paper is structured as follows. Sect. 2 presents a motivating example, followed by related work in Sect. 3. Sect. 4 introduces the T-OCED data model based on TPGs, provide formal definitions, and show how it can be coalesced into an interval-timestamped representation. We then formalize three types of constraints over T-OCED. Sect. 5 demonstrates feasibility through an implementation in Neo4j and shows how semantic violations can be detected. Finally, Sect. 6 discusses the limitations of our approach and outlines future work.

2 Motivating Example

This section introduces a motivating example that illustrates how the lack of explicit temporal semantics in current OCED models leads to inconsistencies and ambiguous interpretations, ultimately preventing reliable analysis.

Consider the process of borrowing books in a library. Individuals may register as members, borrow books for a defined loan period, return or extend loans, incur fines for overdue returns, pay fines, and eventually deregister. A key rule is that a member may deregister only if they currently have no books on loan. The library wishes to analyze whether this rule is respected in practice.

Fig. 1 depicts the snapshot data model underlying this process, i.e., the conceptual structure valid at any moment in time. This model covers all objects, their attributes, and their relationships. Attributes that may change over time are in italic, and snapshot cardinality constraints are shown on the relationships.

To capture the evolution of this process, the library logs timestamped events for books (borrow, return, extend, reserve) and members (register, deregister, fine issued/paid). In OCED, books and members would be objects referenced by these events. However, the resulting object-centric representation would not be aligned with the snapshot data model, because all changes to objects and relationships are aggregated into a single time-agnostic view. Indeed, OCED does not explicitly represent *when* object states (e.g., membership validity) or relationships (e.g,. member-book loan) start and end. The temporal aspects must therefore be reconstructed indirectly from event sequences.

This lack of explicit temporal representation becomes particularly problematic when verifying behavioral rules. Consider the following simplified fragment of the event log: A *Borrow Book* event for Book X by Member A at time t_1, followed by a *Deregister Member* event for Member A at time t_2, with no *Return Book* event recorded in between. To determine whether deregistration is valid, the analyst must *infer* the loan status of Member A at time t_2. Several interpretations are possible: (i) the analyst may assume that the log is complete, in which case the absence of a return implies that the book is still on loan and the deregistration violates the rule; (ii) alternatively, the analyst may assume that events are missing and infer that the book was returned but not logged, making the deregistration valid; or (iii) the analyst may assume that the *Deregister Member* event implicitly closes all open loans, even though this is never stated in

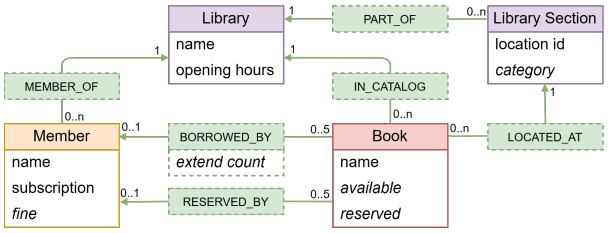

Fig. 1. Snapshot data model of the library with cardinality constraints

the log. Each one of these three interpretations leads to different reconstructions of the member's loan status at time t_2. The log itself cannot resolve this ambiguity because the temporal validity of objects and relationships is not stored explicitly. As a result, analysts must reconstruct object states implicitly. This is an error-prone step that can fundamentally affect the conclusions drawn from the data, particularly in large-scale processes such as baggage-handling systems processing thousands of items per day [9].

This ambiguity remains unresolved in existing data sources or models. They attach time either to events or to objects and relationships, but none provide semantically consistent temporal information at both levels. OCED records time at the event level (cf. Sect 1). In contrast, system logs like database redo logs [26] or SAP change logs timestamp only low-level object or attribute changes, without inherent process context. While the XOC data model [23] conceptually links events to object snapshots, real systems typically record discrete events, not the granular state changes needed to construct such snapshots reliably.

Consequently, we assume that the source of the data is, and remains, events: single-timestamped records with *semantic* meaning that implicitly or explicitly describe changes to objects and their relationships. The challenge is to complement these events with a representation of the *resulting* objects and relationships as they evolve over time. This requires (i) a representation of time periods during which specific objects and relationships hold, and (ii) a unified data model that semantically aligns events with these evolving structures, enabling reasoning over both event occurrences and object states at any moment in time.

3 Related Work

Object-centric process mining requires event data going beyond case-centricity. The first defining requirement, identified in seminal works [29] and shared by all existing proposals, is to equip events with event-to-object (E2O) relationships, thus allowing an event to simultaneously refer to multiple objects, possibly of different types. OCEL supports this feature since version 1 through event tables [16]. EKGs [11] model event data as graphs containing event and object nodes, hence directly supporting E2O relationships through links from event to object nodes. Extensions beyond E2O relationships are much more diversified across proposals. DOCEL [18] extends OCEL1.0 with richer data attributes, supporting multi-valued and time-varying attributes that can be attached to events and objects alike. Time-varying attributes are also supported in OCEL2.0 [20], which further adds the possibility of separately capturing O2O relationships. The latter feature is also naturally present in EKGs, which support links between object nodes. However, in both proposals relationships do not refer to time, leaving it open how they should be interpreted (e.g., as only holding initially, or holding forever, i.e., being *rigid*). Further extensions are tackled in [21] and [31]. While [21] extends OCEL2.0 with object states and their changes over time, [31] does the same starting from DOCEL, and further dealing with the problem of aligning object-centric event data across the whole process lifecycle.

The lack of explicit tracking of how objects and their relationships change over time has led to numerous assumptions or limitations in mining and analysis techniques. Discovery from OCELs has targeted object-centric Petri nets [30] that do not explicitly account for object identity and O2O relationships, leading to underspecified models [30,17] that admit a clear interpretation only when relationships are assumed to be rigid [27]. Similar limitations hold when discovering OC-DECLARE models, which extend the Declare langauge with object-centric features [22]. TOTeM [24] extracts from OCELs interval-based O2O temporal relationships, with two key assumptions: first, that objects are created and destroyed respectively by the first and last events mentioning them; second, that whenever two objects are connected to the same event, then an O2O relationship exists between them, and holds in the time interval where both objects exist.

Approaches for handling time-varying relationships either depend on manually specified reference models with explicit creation, deletion and update constructs [17], or require additional ad-hoc interpretations of the event data [10], since in general it is impossible to unambiguously reconstruct how relationships change from an OCEL or EKG [19].

4 Proposed Data Model

The data model of temporal property graphs (TPGs) [3] captures how objects and relationships *change over time*, but it does not define the concept of an event. We show how to adapt TPGs to object-centric event data by integrating events with the temporal evolution of objects, their attributes, and relationships.

Sect. 4.1 introduces the generic temporal property-graph model and its conversion to an interval-timestamp form. Sect. 4.2 builds on this model to define *Temporal Object Centric Event Data* (T-OCED), which can likewise be converted. Sect. 4.3 formalizes three types of consistency constraints for T-OCED.

4.1 Temporal Property Graphs

A temporal property graph (TPG) extends the notion of a property graph [1,2,14] to include explicit access to time, allowing us to capture how relationships and nodes evolve. TPGs are a theoretical model that explicitly states, for each point in time, whether an object/relation exists and which property values hold for those objects or relations. This point-based approach represents the evolution of a property graph by offering a snapshot at every moment in time. However, timestamping objects at individual time points can lead to considerable space overhead, motivating the use of interval-based representations instead [3].

We first present a formal definition of a temporal property graph (TPG), which can then be transformed into an interval-timestamped temporal property graph (ITPG). In our work, we adopt the TPG definition by Arenas et al. [3].

We consider a label set Λ partitioned into node labels Λ_N and relationship labels Λ_R such that $\Lambda = \Lambda_N \cup \Lambda_R$ and $\Lambda_N \cap \Lambda_R = \emptyset$, and a set of attribute names

Attr whose values come from a domain Val. Temporal property graphs are defined over finite sets of time points, chosen according to the temporal granularity relevant to the application (e.g., seconds, weeks, or years). For convenience, we denote the set of all time points as $\mathbb{N}$, and define a temporal domain Ω which expresses a closed interval of discrete time points between a start time α and an end time β: $\Omega = \{\, i \in \mathbb{N} \mid \alpha \leq i \leq \beta \,\}$, where $\alpha, \beta \in \mathbb{N}$ and $\alpha \leq \beta$.

Definition 1 (Temporal Property Graphs). *A temporal property graph (TPG) $G = (\Omega, N, R, \rho, \lambda, \xi, \#)$ is a graph where*

1. *Ω is a temporal domain; N is a finite set of nodes, R is a finite set of relationships (edges) and $N \cap R = \emptyset$.*
2. *Function $\rho : R \to (N \times N)$ relates a relationsip r to its source and target nodes $\rho(r) = (n_{source}, n_{target})$. We also write $\overrightarrow{r} = (n_{source}, n_{target})$.*
3. *Function $\lambda : (N \cup R) \to 2^{\Lambda}$ assigns to each node $n \in N$ at least one* label *$\lambda(n) \subseteq \Lambda_N$ and $\lambda(n) \neq \emptyset$, and to each relationship $r \in R$ exactly one label $\lambda(r) \in \Lambda_R$, also called relationship type. We also write $N^L = \{\, n \in N \mid \lambda(n) \in L \,\}$ and $R^L = \{\, r \in R \mid \lambda(r) \in L \,\}$.*
4. *Predicate $\xi : (N \cup R) \times \Omega \to \{true, false\}$ states whether a node or relationship $x \in N \cup R$ exists at time t, i.e. $\xi(x,t) = true$. Moreover, if $\xi(r,t)$ and $\rho(r) = (n_1, n_2)$, then $\xi(n_1,t)$ and $\xi(n_2,t)$.*
5. *Partial function $\# : (N \cup R) \times Attr \times \Omega \nrightarrow Val$ states whether attribute a has a value v at time t for a node or relationship $x \in N \cup R$, i.e. $\#(x,a,t) = v$. Moreover, if $\#(x,a,t) = v$ then $\xi(x,t)$. We also write $x.a_t = v$ when $\#(x,a,t) = v$ and $x.a_t = \perp$ if $\#(x,a,t)$ is undefined.*
6. *An attribute a is* static *(i.e. rigid) for a node or relationship $x \in N \cup R$ iff there exists a value v s.t. $\forall t \in \Omega : \xi(x,t) \implies \#(x,a,t) = v$. We write $x.a = v$ when a is a static attribute with value v for x.*

TPGs are subject to two rules that ensure that they align conceptually with sequences of valid conventional property graphs. Specifically, an edge is only permissible at times when both nodes it connects exist (cf. Def. 1.4), and an attribute can only be assigned a value when its associated object exists (cf. Def. 1.5). Furthermore, by restricting $\#(x,a,t)$ to a *finite* collection of tuples (x,a,t), it is ensured that each TPG can be represented finitely.

Interval-timestamped temporal property graphs Each TPG G can be converted into an Interval-timestamped Temporal Property Graph (ITPG) by *coalescing* consecutive time points for which the existence and attribute values remain unchanged. Specifically, an ITPG $I = (\Omega', N, R, \rho, \lambda, \xi', \#')$ that encodes G is characterized as follows: The time domain Ω is substituted by the interval $\Omega' = [\alpha, \beta]$. The components N, R, ρ, λ remain unchanged from G. ξ' is a function assigning each entity $x \in (N \cup R)$ to a set of maximal intervals during which x exists, as per the function ξ. For instance, for $\Omega = \{1,2,3,4,5\}$ and a node n where $\xi(n,1) = \xi(n,2) = \xi(n,3) = \xi(n,5) = true$ and $\xi(n,4) = false$, it results in $\Omega' = [1,5]$ and $\xi'(n) = \{[1,3],[5,5]\}$. We use the open interval notation (a,b) to indicate the discrete interval $[a+1, b-1]$ when convenient. The function $\#'$ is derived from $\#$ in a manner akin to ξ'. The detailed definition is available in [3].

4.2 Temporal Object Centric Event Data

To represent how objects and relationships evolve over time in relation to events, we define a specialized TPG: Temporal Object Centric Event Data (T-OCED), whose structure aligns with the core OCED meta-model [12].

Let OT be a set of object types and RT a set of O2O relationship types. Each relationship type $rt \in RT$ is *strictly typed* by the object types of the nodes it connects. Formally, we denote the typing of rt by $ot(rt) = (ot_1, ot_2) \in OT \times OT$.

In T-OCED, each node is labeled either as an *Event* or as an object type $ot \in OT$. Any event-to-object (E2O) relationship is labeled *observes*. We use $rt \in RT$ to label any O2O relationship.

Definition 2 (Temporal Object Centric Event Data). *A Temporal Object Centric Event Data over object types OT and O2O relationship types RT is a TPG $G = (\Omega, N, R, \rho, \lambda, \xi, \#)$ with node labels $\{Event\} \uplus OT \subseteq \Lambda_N$ and relationship labels $\{observes\} \uplus RT \subseteq \Lambda_R$ with the following properties.*

1. *Every node x is typed with exactly one label $\lambda(x) \in \Lambda_N$.*
2. *For every event node $e \in N^{Event}$, there exists exactly one time point $t \in \Omega$ s.t. $\xi(e,t) = true$. At this time point, the event node e records an activity name $e.act_t \neq \bot$ and a timestamp $e.timestamp_t = t$.*
3. *For every object node $n \in N^{ot}, ot \in OT$, all time points at which n exists are consecutive: for any $t, t' \in \Omega$ with $t < t'$ and $\xi(n,t) = \xi(n,t') = true$, there is no t'' with $t < t'' < t'$ such that $\xi(n,t'') = false$.*
4. *Every observes relationship $r \in R^{observes}$, $\overrightarrow{r} = (e, n)$ is defined from an event node $e \in N^{Event}$ to an object node $n \in N^{ot}, ot \in OT$ s.t. $\xi(e,t)$ iff $\xi(r,t)$ (i.e., r only holds at the time t of event e) and $\xi(r,t)$ implies $\xi(n,t)$ (i.e., n exists at time t).*
5. *Every other relationship $r \in R^{rt}$, $rt \in RT$ (with $ot(rt) = (ot_1, ot_2)$) is defined between two object nodes: $r = (n_1, n_2)$ with $n_1 \in N^{ot_1}, n_2 \in N^{ot_2}$. Additionally, all time points at which r exists are consecutive: for any $t, t' \in \Omega$ with $t < t'$ and $\xi(r,t) = \xi(r,t') = true$, there is no t'' with $t < t'' < t'$ such that $\xi(r,t'') = false$.*

We denote all object nodes by $N^{OT} = \bigcup_{ot \in OT} N^{ot}$ and all O2O relationships by $R^{RT} = \bigcup_{rt \in RT} R^{rt}$.

T-OCED is subject to several rules reflecting how events and objects behave in real processes. In T-OCED, events are instantaneous (occur at a single point in time), whereas objects and O2O relationships persist over a single continuous time interval, during which their type remains static – even though, in reality, objects or relationships may disappear, reappear, or change type (e.g., a library member becoming an employee). However, these limitations do not reduce modeling expressiveness: any type change can be represented by modeling the same real-world object as multiple object nodes, or the same O2O relationship as multiple edges, each with its own lifetime and type.

Interval-timestamped Temporal Object Centric Event Data Since every T-OCED instance is a specialization of a TPG, these graphs can be coa-

lesced into interval-timestamped TPGs using the operation defined in Sect. 4.1. As T-OCEDs constrain TPGs (Def. 2), the corresponding IT-OCED G' of an T-OCED has the following properties[1]:

1. The existence interval for event nodes $e \in N^{Event}$ is a single point: $\xi'(e) = \{[t, t]\}$ for exactly one time point $t \in \Omega'$.
2. The existence interval for object nodes $n \in N^{OT}$ is a consecutive interval: $\xi'(n) = \{[a, b]\}$ for some $a \leq b$ with $a, b \in \Omega'$.
3. The existence interval for observes relationships $r \in R^{observes}$ is a single point: $\xi'(r) = \{[t, t]\}$ where $t \in \Omega'$ is the timestamp of the corresponding event $e \in N^{Event}$ with $\xi'(e) = \{[t, t]\}$ and $r = (e, n)$.
4. The existence interval for every O2O relationships $r \in R^{RT}$ is a consecutive interval: $\xi'(r) = \{[a, b]\}$ for some $a \leq b$ with $a, b \in \Omega'$.

Fig. 2 illustrates an instance of IT-OCED for the library process described in Sect. 2. It depicts several events related to members *Alice*, *Lewis*, and the book *Alice in Wonderland*, along with their validity intervals and the intervals of the related relationships and attributes.

4.3 Constraints

To maintain semantic consistency between events and evolving objects and relationships in T-OCED, we introduce three kinds of constraints: *key constraints* ensuring unique identification of nodes, *cardinality constraints* governing the number of permitted relationships, and *semantic constraints* ensuring that events correspond to valid changes in objects, attributes, and relationships.

Key Constraints In a property graph, a node already has an inherent unique identity, i.e., the node itself. However, many applications designate certain node

[1] The full definition can be found in App. A (see: `https://anonymous.4open.science/r/IT-OCED-Library-7AB4`)

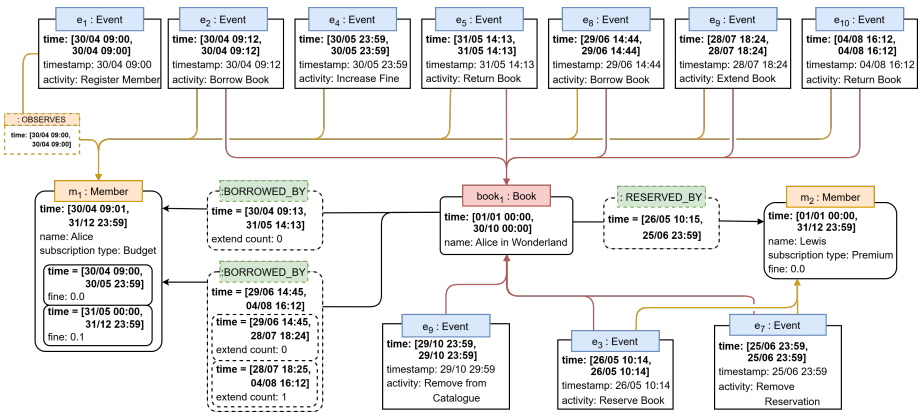

Fig. 2. An instance of IT-OCED showing a small fragment of the library process.

attributes as *external identifiers* that must be unique across all instances of a given type. Such attributes are called *keys* and they must be *static*, which we formalize as key constraints.

Node key constraints Let G be an T-OCED and $K \subseteq Attr$ be a nonempty set of attribute names. We say that K is a key for type $l \in \Lambda_N$ if, for any two nodes $x_1, x_2 \in N^l$, $(\forall a \in K : \ x_1.a = x_2.a) \ \Rightarrow x_1 = x_2$.

For relationships, the edge between two nodes already provides an inherent unique identity, so defining keys for relationships is usually unnecessary. In one-to-one and one-to-many relationships (cf. see 4.3 Cardinality Constraints), the edge is typically structural: it expresses how two objects are related, but the relationship itself does not have an independent identity. Examples include a book being borrowed by at a member, or a book located at a section in the library. In such cases, assigning keys to relationships offers no modelling benefits.

In contrast, many-to-many relationships often represent an underlying business object [15]. Here, the relationship carries meaning beyond simply connecting two objects, and is therefore commonly reified as its own object. For instance, a member holding memberships at multiple libraries correspond to a *Membership* object with its own identity and attributes (e.g., membership type). In these cases, the relationship should be reified as an object node, after which its attributes can be treated as keys like those of any other object.

Cardinality Constraints We may wish to constrain the cardinality of relationship types between object nodes of a given type (e.g., one-to-one, one-to-many, or many-to-many). Adding time makes cardinality more nuanced: a relationship can be one-to-many at *every moment*, even if multiple such relationships occur over time on the 'one' side. For instance, a book may be borrowed by at most one member at any moment, yet over its lifetime be borrowed many times. To capture this distinction, we differentiate between two kinds of cardinality constraints: snapshot and lifespan cardinality.

Let G be an T-OCED and $rt \in RT$ be a relationship type with $ot(rt) = (ot_1, ot_2)$.

1. **Snapshot cardinality constraints** describe how many relationships of a given type may exist simultaneously and should hold at every time point.
   Let $card_{min}, card_{max} \in \mathbb{N} \cup \{\infty\}$ be cardinality bounds. We say that relationship type rt has snapshot cardinality $[card_{min}, card_{max}]$ iff: for every time point $t \in \Omega$ and every object node $n_1 \in N^{ot_1}$ and $\xi(n_1, t)$, then $card_{min} \leq |\{\, r = (n_1, n_2) \in R^{rt} \mid n_2 \in N^{ot_2}, \xi(r, t) \,\}| \leq card_{max}$, i.e., at any valid time point t, an object of type ot_1 must be connected to between $card_{min}$ and $card_{max}$ objects of type ot_2 through relationships of type rt that are valid at that time t.

2. **Lifespan cardinality constraints** describe how many relationships may occur in total across time and apply to the entire lifetime of an object (e.g., a book may be borrowed at most 10 times during their life).

Let $life_{min}, life_{max} \in \mathbb{N} \cup \{\infty\}$ be lifetime bounds. We say that relationship type rt has lifespan cardinality $[life_{min}, life_{max}]$ iff: for every object node $n_1 \in N^{ot_1}$, $life_{min} \leq |\{\, r = (n_1, n_2) \in R^{rt} \mid n_2 \in N^{ot_2} \,\}| \leq life_{max}$. Note that the lifespan cardinality constraint has no temporal constraint $\xi(r, t)$. Thus, over its entire lifetime, an object of type ot_1 must participate in between $life_{min}$ and $life_{max}$ relationships of type rt with objects of type ot_2 regardless of when those relationships occur.

Semantic Constraints Semantic constraints capture the expected relationships between events and the objects and relationships they affect. For example, *Borrow Book* updates a `Book` object (updating *available*) and creates a `BORROWED BY` relationship to a `Member`. We base these constraints on CRUD operations (Create, Read, Update, and Delete), the fundamental operations on persistent data [25]. To apply them, we must know which operation each activity performs. Because event operation semantics are often not explicitly represented in existing data models, we assume an oracle that provides this information, consistent with the OCED Core Meta-Model [12], where known activity semantics may be expressed as a qualifier on the E2O *observes* relationship.

For relationships, we introduce an additional *Replace* operator. Since relationships can be seen as non-rigid attributes (similar to how ER relationships are implemented via foreign keys) they may be *updated* from an object's perspective. From the relationship's perspective, however, such a change corresponds to deleting the old relationship and creating a new one. *Replace* therefore subsumes both *Create* and *Delete*, capturing activities that remove an existing relationship by introducing a new one. For example, a *Move Book* activity replaces the `LOCATED AT` relationship between a `Book` and its current `Library Section` with a new one pointing to the updated one. Importantly, *Replace* applies only to relationships, not objects, since object identity persists across updates, whereas relationship identity changes when an endpoint changes.

We consider an IT-OCED G'[2], a finite set of activity names Act, a finite set of object types OT and a finite set of relationship types RT and an operator set $OP = \{Create, Read, Update, Delete, Replace\}$.

Definition 3 (Oracle Function). *Partial function $f : Act \times (OT \cup RT) \nrightarrow OP$ assigns to an activity name $a \in Act$ and type $tp \in OT \cup RT$ an operation type $op \in OP$. We also write $f_{tp}(a) = op$ as shorthand. Additionaly, $f_{ot}(a) \neq Replace$ for any $ot \in OT$.*

Using the Oracle function, we can define the semantic constraints.

Semantic Constraints on Events and Objects Let $e \in N^{\{Event\}}$ be an event occurring at time $t \in \Omega$, i.e. $\xi(e) = \{[t, t]\}$. Let $ot \in OT$. If the oracle assigns $f_{ot}(e.act) \neq \perp$, then the following must hold.

[2] For readability, we state the constraints over IT-OCED; an equivalent formulation over T-OCED appears in App. B (see: `https://anonymous.4open.science/r/IT-OCED-Library-7AB4`)

- **CREATE** If $f_{ot}(e.act) = Create$, then there exists an object $n \in N^{ot}$ such that $(e, n) \in R^{observes}$ and $\xi(n) = \{(t, b]\}$ for some $b \geq t$, i.e. e causes an object n of type ot to come into existence immediately after time t and to remain in existence until (and including) time b.
- **READ** If $f_{ot}(e.act) = Read$, then there exists an object $n \in N^{ot}$ such that $(e, n) \in R^{observes}$ and $\xi(n) = \{[a, b]\}$ for some $a \leq t \leq b$, i.e. event e does indeed read an object of type ot.
- **UPDATE** If $f_{ot}(e.act) = Update$, then there exists an object $n \in N^{ot}$ such that $(e, n) \in R^{observes}$, and $\xi(n) = \{[a, b]\}$ for some $a \leq t \leq b$, and there exists an attribute $a \in Attr$ such that $n.a_t \neq n.a_{t+1}$, i.e. an update on an object n of type ot is observable as as attribute value change at time t.
- **DELETE** If $f_{ot}(e.act) = Delete$, then there exists an object $n \in N^{ot}$ such that $(e, n) \in R^{observes}$ and $\xi(n) = \{[a, t]\}$ for some $a \leq t$, i.e. Hence, e marks the end of the validity interval of object n of type ot, which ceases to exist immediately after time t.

When formulating these constraints, it is crucial to interpret an event's timestamp and determine when its effects take place. In this paper, we assume that an event's effect becomes visible *immediately after* the event, i.e., an attribute updated at time t takes effect at $t + 1$, and the old value still holds at t. This addresses the well-known *divided instant problem* [13], which concerns the ambiguity of the exact moment of change and requires ensuring that *only one* value is valid at any given instant. This assumption also implies that an object will only come into existence just after a CREATE event, and that it still exists at the time of a DELETE event, ceasing to exist only one instant later. This interpretation aligns with common reasoning in event calculus [8].

Because the divided instant problem forces a design choice about when effects occur, one could alternatively decide that effects take place *at* the timestamp of the event, which would imply, for example, that an object no longer exists at the time of deletion. Either choice requires some flexibility in the TPG definition. In a TPG, a relationship may exist only when both endpoints exist, but under our assumption, during create the object does not yet exist. Therefore, for the *observes* relationship, we relax this constraint to allow such transitional cases.

Semantic Constraints on Events and Relationships Since *observes* relationships relate events to object nodes, and not directly to O2O relationships, the semantic constraints for relationships must ensure that an event observes at least one object node n to which the relevant relationship is *incident*. We say that an O2O relationship is incident to an object node n if $\overrightarrow{r} = (n, n') \in R^{RT}$ or $\overrightarrow{r} = (n', n) \in R^{RT}$.

Let $e \in N^{Event}$ be an event occurring at time $t \in \Omega$, i.e. $\xi(e) = \{[t, t]\}$. Let $rt \in RT$. If the oracle assigns $f_{rt}(e.act) \neq \perp$, then the following must hold.

- **CREATE** If $f_{rt}(e.act) = Create$, then there exist an object $n \in N^{OT}$ such that $(e, n) \in R^{observes}$, $r \in R^{rt}$ is incident to n and $\xi(r) = \{(t, b]\}$ for some $b \geq t$, i.e. e causes a r of type rt incident to the observed object n to come into existence immediately after time t and continues to exist until (and including) a time b.

- **READ** If $f_{rt}(e.act) = Read$, then there exist an object $n \in N^{OT}$ such that $(e, n) \in R^{observes}$, $r \in R^{rt}$ is incident to n and $\xi(r) = \{[a, b]\}$ with $a \leq t \leq b$, i.e. event e does indeed read a relationship r of type rt that exists at time t.
- **UPDATE** If $f_{rt}(e.act) = Update$, then there exist an object $n \in N^{OT}$ such that $(e, n) \in R^{observes}$, $r \in R^{rt}$ is incident to n, $\xi(r) = \{[a, b]\}$ with $a \leq t \leq b$, and there exists an attribute $attr \in Attr$ such that $r.attr_t \neq r.attr_{t+1}$, i.e. an update on r of type rt incident to the observed node n is observable as as attribute value change at time t.
- **DELETE** If $f_{rt}(e.act) = Delete$, then there exist an object $n \in N^{OT}$ such that $(e, n) \in R^{observes}$, $r \in R^{rt}$ is incident to n and $\xi(r) = \{[a, t)\}$ for some $a \leq t$, i.e. r of type rt incident to the observed node n exists exactly from time a onwards and continues to exist until exactly time t.
- **REPLACE** If $f_{rt}(e.act) = Replace$, then there exist an object $n \in N^{OT}$ such that $(e, n) \in R^{observes}$, $r_{old} \in R^{rt}$ is incident to n, $r_{new} \in R^{rt}$ is incident to n (with $r_{old} \neq r_{new}$), $\xi(r_{old}) = \{[a, t]\}$, $\xi(r_{new}) = \{(t, b]\}$ with $a \leq t \leq b$, i.e., a relationship update of type rt for the observed object n is represented by terminating r_{old} at time t (so it no longer exists at $t + 1$) and initiating r_{new} at time $t + 1$.

From the semantic constraints, it also naturally follows that if an object n of type ot is observed by an event e with $f_{ot}(e.act) = Create$, there cannot exist any ealier event e' such that $e'.timestamp < e.timestamp$ and $(e', n) \in R^{observes}$. If such an event e' would exist, it would imply that n is observed by e' at time $e'.timestamp$, but by the Create constraint, the object n does not yet exist at any time before $e.timestamp$. Correspondingly, if there is a *Delete* event e recorded for n, no event may refer to n at any time later than $e.timestamp$. Moreover, any *Read* and *Update* event must be after the *Create* event and before the *Delete* event, if such events have been recorded.

We also observe that the *Cardinality Constraints* have implications for the *Semantic Constraints*. For example, if a snapshot cardinality constraint specifies that every `Book` must be in the catalogue of exactly one `Library` (i.e. a many-to-one relationship), then whenever an activity creates an `Book`, it must also create the corresponding `IN CATALOGUE OF` relationship to ensure the cardinality constraint is satisfied at that moment in time.

5 Demonstration

We demonstrate the feasibility of our proposed data model using available graph databases[3]. Since no widely used graph database offers native temporal support, we implemented our model in Neo4j, making the following representation choices for the coalesced IT-OCED: We chose to represent object evolution using global state nodes, with all relationships attached to the object nodes with validity intervals expressed as `validFrom` and `validTo` properties. Relationship evolution

[3] The implementation is available on `https://anonymous.4open.science/r/IT-OCED-Library-7AB4`

is captured through multiple relationship edges; a relationship counter tracks which edges represent the same conceptual relationship over time. Exploring the performance and conceptual implications of these decisions and alternatives (e.g., multiple object nodes, relations attached to states or object nodes) is beyond the scope of this paper and remains future work.

To generate an IT-OCED instance in line with our model decisions and subsequently demonstrate verifiability of constraints over this data model, we generated a simulated dataset of the library process described in Section 2. The dataset takes the form of a raw event table that records each activity together with its timestamp, the identifier of the involved member, its outstanding fine, and its subscription type. It also includes the identifiers of up to five observed books along with their availability or reservation status, and the identifier of the library. We injected controlled violations to test whether queries implementing these constraints could correctly identify them.

We then used OCED-PG [28] to translate the simulated data (event table) into an IT-OCED instance in line with our model decisions, thereby mapping event data into events, objects, and relationships. Events were assigned their validity intervals directly from their timestamps. For objects and relationships, we derived when they were created, updated and deleted based on the known semantics of the activity. For example, a *Register Member* event occurring at time t observing member m creates member m at time t. For initial states, i.e., before the first observed event for an object, we inferred values where possible. For instance, if the first event for a book is *Reserve Book*, we infer that its initial value for *reserved* was *false*. Objects and relationships without an explicit create or DELETE event were assumed to hold from the start of the dataset until its end.

To evaluate where the constructed IT-OCED instance satisfies or violates the formal IT-OCED specification as well as the cardinality and semantic constraints, we translated all constraints to Cypher queries. The following example illustrates the translation principle; the implementation includes the full set of constraints together with their corresponding queries. The following Cypher query implements the instantiation of the UPDATE constraint for *Extend Book* of our running example, where *Extend Book* updates the `:BORROWED_BY` relationship by modifying the attribute `extendCount`. The query detects whether any feature of the relationship has changed at the timestamp of the event:

```
1    MATCH (e:Event {activity: 'Extend Book'})
2    RETURN EXISTS{
3     (e) - [:OBSERVES] -> (b:Book) - [r:BORROWED_BY] -> (:Member) <- [prev:BORROWED_BY] - (b)
4     WHERE prev.validTo = e.timestamp AND r.validFrom = e.timestamp
5     AND any(k IN keys(r) + keys(prev) WHERE
6        NOT k in ['validFrom', 'validTo'] //ignore temporal keys
7        AND properties(prev)[k] <> properties(r)[k]) // detect changed feature
8     }, count(e) as eventCount
```

When the specific feature to be updated is known, the generic comparison (lines 5-7) can be replaced with a direct value comparison. In our case, this simplifies to `AND r.extendCount <> prev.extendCount`.

Simulated Period	#Events	#Members	#Books	#observes	#O2O relation	Build Time	CC Time
1 year	170,072	1216	7000	216,231	46,374	1.3 min	0.2 min
2 year	309,995	1242	7000	410,596	81,960	2.2 min	0.5 min
3 year	470,063	1258	7000	621,469	114,839	3.8 min	1.0 min

Table 1. Simulation Size and Performance Metrics per Period Length

To cover all types of constraints in a non-trivial setting, we generated datasets for a single library operating over several time periods, each preceded by a 50-day warm-up phase to simulate an already functional environment. For each period, we constructed an IT-OCED instance in Neo4j v5.26, using non-optimized queries and indexing only on unique identifiers for events and objects, as well as on the validity intervals of State nodes. The resulting performance measurements are reported in Tab. 1. For each simulation period, we recorded the time required to build the IT-OCED instance and the time needed to check all constraints (CC). In total, we defined 22 semantic constraints, 6 snapshot cardinality constraints, and 1 lifetime constraint.

Because of how the IT-OCED instance is constructed, some semantic constraints (e.g., each CREATE event producing an object) always hold. However, when CREATE or DELETE events are missing, other constraints (e.g., snapshot cardinality) are inevitably violated because the object's lifecycle becomes inconsistent. Certain semantic violations, such as missing updates, remain detectable since the expected value change is absent. Even when some semantic constraints cannot be violated, they remain useful for validating data transformation.

6 Conclusion

This work introduces the data model T-OCED, based on Temporal Property Graphs (TPGs), to explicitly represent time for events, objects, and their relationships. By defining three types of constraints, we ensure semantic alignment between events and evolving object states. We demonstrated feasibility by implementing the model in Neo4j and validating it on a simulated library dataset with controlled violation injections.

This work forms a first step toward jointly modeling events and evolving structures while ensuring semantic alignment, providing a solid foundation for process discovery and conformance checking techniques that rely on temporal synchronization. Limitations remain, however. Because events are our only data source, some object states cannot be inferred, which also restricts certain constraint checks to the observed timeframe. When snapshots are available, however, T-OCED offers a structured way to combine them with events. Furthermore, as current graph databases lack native temporal support, implementation choices must be made; evaluating alternative representations is left for future work. Finally, while TPGs support temporal navigation over objects and relationships [3], they lack navigators for event sequences similar to Directly Follows Relationships used in EKGs [11], which we identify as another direction for future research.

References

1. Angles, R., Arenas, M., Barceló, P., Boncz, P., Fletcher, G., Gutierrez, C., Lindaaker, T., Paradies, M., Plantikow, S., Sequeda, J., et al.: G-core: A core for future graph query languages. In: SIGMOD 2018. pp. 1421–1432. ACM (2018)
2. Angles, R., Arenas, M., Barceló, P., Hogan, A., Reutter, J., Vrgoč, D.: Foundations of modern query languages for graph databases. ACM Computing Surveys (CSUR) **50**(5), 1–40 (2017)
3. Arenas, M., Bahamondes, P., Aghasadeghi, A., Stoyanovich, J.: Temporal regular path queries. In: ICDE 2022. pp. 2412–2425 (2022)
4. Artale, A., Franconi, E.: Foundations of temporal conceptual data models. In: Conceptual Model. Found. Appl. vol. 5600, pp. 10–35. Springer (2009)
5. Barenholz, D., Montali, M., Polyvyanyy, A., Reijers, H.A., Rivkin, A., van der Werf, J.M.E.: There and back again: on the reconstructability and rediscoverability of typed jackson nets. In: PETRI NETS 2023. pp. 37–58. Springer (2023)
6. Berti, A., Montali, M., van der Aalst, W.M.P.: Advancements and challenges in object-centric process mining: A systematic literature review. CoRR **abs/2311.08795** (2023)
7. Chaki, R., Calvanese, D., Montali, M.: Generation of timelines from event knowledge graphs using domain knowledge. In: CISIM. LNCS, vol. 15927, pp. 275–289. Springer (2025)
8. Chesani, F., Mello, P., Montali, M., Torroni, P.: A logic-based, reactive calculus of events. Fundamenta Informaticae **105**(1-2), 135–161 (2010)
9. Chu, V.: Using event knowledge graphs to model multi-dimensional dynamics in a baggage handling system. Master's thesis, Eindhoven University of Technology (2022)
10. van Detten, J.N., Schumacher, P., Leemans, S.J.J.: Modeling and discovering dynamic identity relations in object-centric process mining. In: ICPM. pp. 1–8. IEEE (2025)
11. Fahland, D.: Process mining over multiple behavioral dimensions with event knowledge graphs. In: Process Mining Handbook, vol. 448, pp. 274–319. Springer (2022)
12. Fahland, D., Montali, M., Lebherz, J., van der Aalst, W.M.P., van Asseldonk, M., Blank, P., Bosmans, L., Brenscheidt, M., di Ciccio, C., Delgado, A., et al.: Towards a simple and extensible standard for object-centric event data (OCED)–core model, design space, and lessons learned. arXiv preprint arXiv:2410.14495 (2024)
13. Fisher, M.D., Gabbay, D.M., Vila, L.: Handbook of temporal reasoning in artificial intelligence, vol. 1. Elsevier (2005)
14. Francis, N., Green, A., Guagliardo, P., Libkin, L., Lindaaker, T., Marsault, V., Plantikow, S., Rydberg, M., Selmer, P., Taylor, A.: Cypher: An evolving query language for property graphs. In: Proc. of the 2018 Int. Conf. on Management of Data. pp. 1433–1445 (2018)
15. Frisendal, T.: Getting the Structure Right, pp. 45–60. Apress, Berkeley, CA (2018)
16. Ghahfarokhi, A.F., Park, G., Berti, A., van der Aalst, W.M.P.: Ocel: A standard for object-centric event logs. In: New Trends in Database and Information Systems. pp. 169–175. Springer (2021)
17. Gianola, A., Montali, M., Winkler, S.: Object-Centric Conformance Alignments with Synchronization. In: CAiSE 2024. vol. 14663, pp. 3–19 (2024)
18. Goossens, A., De Smedt, J., Vanthienen, J., van der Aalst, W.M.P.: Enhancing data-awareness of object-centric event logs. In: ICPM 2022 Workshops. vol. 468, pp. 18–30. Springer (2022)

19. Hooshyar, H., Fumagalli, M., Montali, M., Guizzardi, G.: Time and relations into focus: Ontological foundations of object-centric event data. CoRR **abs/2512.14425** (2025)
20. Koren, I., Adams, J.N., Berti, A., van der Aalst, W.M.P.: Ocel 2.0 resources – www.ocel-standard.org. In: ICPM 2023 Demo Track (2023)
21. Kretzschmann, D., Berti, A., van der Aalst, W.M.P.: State-aware object-centric process mining: Enhancing OCEL 2.0 with explicit state transitions. In: EDOC 2025. vol. 16213, pp. 103–118. Springer (2025)
22. Küsters, A., van der Aalst, W.M.P.: OC-DECLARE: discovering object-centric declarative patterns with synchronization. In: BPM 2025. vol. 16044, pp. 162–179. Springer (2025)
23. Li, G., de Murillas, E.G.L., de Carvalho, R.M., van der Aalst, W.M.P.: Extracting object-centric event logs to support process mining on databases. In: CAiSE Forum 2018. pp. 182–199 (2018)
24. Liss, L., Adams, J.N., van der Aalst, W.M.P.: Totem: Temporal object type model for object-centric process mining. In: BPM 2024 (2024)
25. Martin, J.: Managing the data base environment. Prentice Hall PTR (1983)
26. González López de Murillas, E., Hoogendoorn, G., Reijers, H.A.: Redo log process mining in real life: Data challenges & opportunities. In: BPM 2017 Workshops. pp. 573–587. Springer (2017)
27. Seidel, A., Winkler, S., Gianola, A., Montali, M., Weske, M.: To bind or not to bind? Discovering stable relationships in object-centric processes. In: ER 2025. vol. 16189, pp. 223–241 (2025)
28. Swevels, A., Fahland, D., Montali, M.: Implementing object-centric event data models in event knowledge graphs. In: ICPM 2023 Workshops. pp. 431–443. Springer (2023)
29. van der Aalst, W.M.P.: Object-centric process mining: Dealing with divergence and convergence in event data. In: SEFM 2019. vol. 11724, pp. 3–25. Springer (2019)
30. van der Aalst, W.M.P., Berti, A.: Discovering Object-centric Petri Nets. Fundamenta Informaticae **175**(1-4) (2020)
31. Verbruggen, C., Goossens, A., De Smedt, J., Vanthienen, J., Snoeck, M.: iDOCEM: defining a common terminology for object-centric event logging and data-centric process modelling. Software & Systems Modeling **24**(1) (2025)

A Definition Interval-timestamp Temporal Object Centric Event Data(IT-OCED)

Definition 4 (Interval-timestamp Temporal Object Centric Event Data). *An interval-timestamp Temporal Object Centric Event Data (IT-OCED) is an ITPG* $I = (\Omega, N, R, \rho, \lambda, \xi, \#)$ *with node labels* $\{Event\} \uplus OT \subseteq \Lambda_N$*, where* OT *is the set of object types, and relationship labels* $\{observes\} \uplus RT \subseteq \Lambda_R$*, where* RT *is the set of O2O relationship types, with the following properties.*

1. *Every node* x *is typed with exactly one label* $\lambda(x) \in \Lambda_N$
2. *For every event node* $e \in N^{Event}$*, the existence interval is a single point:* $\xi(e) = \{[t, t]\}$ *for a time point* $t \in \Omega$*. At this time point, the event node* e *records a static activity name* $e.act \neq \perp$ *and a timestamp* $e.timestamp = t$*.*
3. *For every object node* $n \in N^{OT}$*, the existence interval is a consecutive interval:* $\xi(n) = \{(a, b]\}$ *for some* $a \leq b$ *with* $a, b \in \Omega$*.*
4. *Every observes relationship* $r \in R^{observes}$*,* $\overrightarrow{r} = (e, n)$ *is defined from an event node to an object node,* $e \in N^{Event}$*,* $n \in N^{OT}$ *s.t.* $\xi(e) = \{[t, t]\}$ *iff* $\xi(r) = \{[t, t]\}$ *and* $\xi(n) = \{(a, b]\}$ *with* $a \leq t \leq b$*.*
5. *Every other relationship* $r \in R^{RT}$ *is defined between two object nodes:* $r = (n_1, n_2)$ *with* $n_1, n_2 \in N^{OT}$*. Additionally, its existence interval is a consecutive interval:* $\xi(n) = \{(a, b]\}$ *for some* $a \leq b$ *with* $a, b \in \Omega$*.*

B Semantic Constraints defined over T-OCED

Semantic Constraints on Events and Objects (over T-OCED)

Let $e \in N^{Event}$ be an event occurring at time $t \in \Omega$, i.e. $\xi(e, t)$. Let $ot \in OT$. If the oracle assigns $f_{ot}(e.act) \neq \perp$, then the following must hold.

- **CREATE** If $f_{ot}(e.act) = Create$, then there exists an object $n \in N^{ot}$ such that $(e, n) \in R^{observes}$, $\xi(n, t{+}1)$ and for all $t' \in \Omega : t' \leq t \Rightarrow \xi(n, t') = false$, i.e. e causes an object n of type ot to come into existence immediately after time t.
- **READ** If $f_{ot}(e.act) = Read$, then there exists an object $n \in N^{ot}$ such that $(e, n) \in R^{observes}$ and $\xi(n, t)$, i.e. event e does indeed read an object of type ot.
- **UPDATE** If $f_{ot}(e.act) = Update$, then there exists an object $n \in N^{ot}$ such that $(e, n) \in R^{observes}$, $\xi(n, t)$ and there exists an attribute $a \in Attr$ such that $n.a_t \neq n.a_{t+1}$, i.e., an update on an object n of type ot is observable as an attribute value change at time t.
- **DELETE** If $f_{ot}(e.act) = Delete$, then there exists an object $n \in N^{ot}$ such that $(e, n) \in R^{observes}$, $\xi(n, t)$ and for all $t' \in \Omega : t' > t \Rightarrow \xi(n, t') = false$, i.e. n of type ot stops existing exactly from time t onwards.

Semantic Constraints on Events and Relationships (over T-OCED)

Let $e \in N^{Event}$ be an event occurring at time $t \in \Omega$, i.e. $\xi(e, t) = true$. Let $rt \in RT$. If the oracle assigs $f_{rt}(e.act) \neq \perp$, then the following must hold.

- **CREATE** If $f_{rt}(e.act) = Create$, then there exist an object $n \in N^{OT}$ such that $(e, n) \in R^{observes}$, $r \in R^{rt}$ is incident to n and $\xi(r, t + 1) = true$ and

for all $t' \in \Omega : t' \leq t \Rightarrow \xi(r, t') = false$, i.e. e causes r of type rt incident to the observed node n_1 to come into existence immediately after time t.

- **READ** If $f_{rt}(e.act) = Read$, then there exist an object $n \in N^{OT}$ such that $(e, n) \in R^{observes}$, $r \in R^{rt}$ is incident to n and $\xi(r, t) = true$, i.e. event e does indeed read a relationship of type rt incident to node n_1 at time t.
- **UPDATE** If $f_{rt}(e.act) = Update$, then there exist an object $n \in N^{OT}$ such that $(e, n) \in R^{observes}$, $r \in R^{rt}$ is incident to n, $\xi(r, t) = true$ and there exists an attribute $a \in Attr$ such that $r.a_t \neq r.a_{t+1}$, i.e., an update on a relationship of type rt incident to node n_1 is observable as an attribute value change at time t.
- **DELETE** If $f_{rt}(e.act) = Delete$, then there exist an object $n \in N^{OT}$ such that $(e, n) \in R^{observes}$, $r \in R^{rt}$ is incident to n, $\xi(r, t) = true$ and for all $t' \in \Omega : t' > t \Rightarrow \xi(r, t') = false$, i.e. r of type rt incident to the observed node n_1 still exists at time t and stops existing from time t onwards.
- **REPLACE** If $f_{rt}(e.act) = Replace$, then there exist an object $n \in N^{OT}$ such that $(e, n) \in R^{observes}$, $r_{old} \in R^{rt}$ is incident to n, $r_{new} \in R^{rt}$ is incident to n (with $r_{old} \neq r_{new}$), $\xi(r_{old}, t) = true$, $\xi(r_{new}, t + 1) = true$ for all $t' \in \Omega : t' > t \Rightarrow \xi(r_{old}, t') = false$, for all $t'' \in \Omega : t'' \leq t \Rightarrow \xi(r_{new}, t'') = false$, i.e., a relationship update of type rt for the observed object n is represented by terminating r_{old} at time t (so it no longer exists at $t + 1$) and initiating r_{new} at time $t + 1$.
  stops existing exactly from time t onwards.